Esercitazione 15 aprile

Classi e allocazione dinamica

[Esercizio C069] Classe ContoBancario V2.0

Si progetti e implementi una classe *ContoBancario* che rappresenti un conto corrente con seguenti attributi:
char* *intestatario* (allocato dinamicamente), int *numeroConto*, float *saldo*

- Implementare un costruttore di default che inizializzi: intestatario a nullptr numeroConto a 0 saldo a 0.0
- Implementare un costruttore parametrico che allochi dinamicamente la memoria per intestatario
- Implementare un costruttore di copia che realizzi una deep copy del campo intestatario
- Implementare un distruttore che liberi correttamente la memoria allocata
- Implementare i seguenti metodi setter:
  - setIntestatario(const char*), setNumeroConto(int), setSaldo(float) Il saldo non può essere negativo
- Implementare metodi getter per tutti gli attributi
- Implementare il metodo: void trasferisci con due parametri: *importo* e *commissione*
  - il metodo deve sottrarre dal saldo l'importo più una *commissione*
  - la commissione ha valore di default pari a 1.0
- Implementare il metodo: void *applicaInteresse* con un parametro *percentuale*
  - il metodo deve aumentare il saldo applicando una percentuale di interesse
  - la percentuale ha valore di default pari al 2%
- Implementare un metodo: void *stampa*() per visualizzare i dati del conto

Nel main():

- Creare un oggetto *ContoBancario* con costruttore parametrico
- Effettuare: A) un trasferimento utilizzando il valore di default della commissione, B) un trasferimento specificando anche la commissione, C) Applicare l'interesse senza specificare la percentuale
- Stampare i dati finali del conto

[Esercizio C066] Classe Studente

Si progetti e implementi una classe Studente che rappresenti uno studente universitario con seguenti attibuti: char* *nome* (stringa allocata dinamicamente), int *matricola*, float *media*:

- Implementare un costruttore di default che inizializzi degli attibuti usando lista di inizializzazione
- Implementare un costruttore parametrico che:
  - riceva come parametri una stringa (const char*), un intero e un float
  - utilizzi la lista di inizializzazione per inizializzare i membri non dinamici
  - allochi dinamicamente la memoria per nome e ne copi il contenuto
- Implementare un distruttore che liberi correttamente la memoria allocata
- Implementare il metodo setter *setNome* deve gestire correttamente la memoria (deallocazione e nuova allocazione)
  Implementare il metodo setter *setVoti* che prende un array dei voti ed il numero e aggiorna entrambi attributi
- Implementare i metodi getter per tutti i attributi
- Implementare un metodo: void stampa() per visualizzare i dati dello studente

Nel main():

- creare un oggetto con costruttore parametrico
- creare un oggetto come copia di un altro
- creare un oggetto con costruttore di default e inizializzarlo tramite setter
- stampare i dati degli oggetti

[Esercizio C067] Classe Corso

Si progetti e implementi una classe Corso che rappresenti un esame universitario. La classe deve contenere i seguenti attributi: char* *nome* (stringa allocata dinamicamente), int *voto*

- Implementare un costruttore di default usa la lista di inizializzazione,
- Implementare un costruttore parametrico che usa la lista di inizializzazione per attributi non dinamici,
  - allochi dinamicamente la memoria per nome,
- Implementare un costruttore di copia che realizzi una deep copy del attibuto *nome,*
- Implementare un distruttore che liberi correttamente la memoria allocata,
- Implementare il metodi setter:
  setVoto deve verificare che il voto sia compreso tra 18 e 30 in caso contrario, impostare il voto a 18,
  setNome che che dealloca fa la nuova allocazione di memoria,
- Implementare i metodi getter per tutti i attibuti,
- Implementare un metodo void stampa() per visualizzare i dati del corso.

Nel main():

- Chiedere all'utente quanti corsi vuole inserire,
- Allocare dinamicamente un array di oggetti Corso,
- Per ogni corso: leggere da input il nome, leggere il voto impostare i dati tramite setter,

- Calcolare e stampare la media dei voti.

[Esercizio C068] Classi Corso e Studente insieme

Dato l'implementazione della classe *Corso* come nel esercizio precedente, si implementi una classe Studente con i seguenti attributi: char* *nome* (allocato dinamicamente), int *numCorsi*, Corso** *corsi* - array dinamico di puntatori a oggetti Corso

- Implementare un costruttore di default,
- Implementare un costruttore parametrico che:
  - A) allochi dinamicamente l'array di puntatori a Corso, B) inizializzi i puntatori a nullptr
- Implementare un distruttore che:
  - A) liberi la memoria del *nome*, B) deallochi tutti gli oggetti Corso, C) liberi l'array di puntatori
- Implementare tutti i getter,
- Implementare un setter: *setCorso(int index, const char* nomeCorso, int voto)*
  - il metodo deve creare dinamicamente un oggetto Corso e inserirlo nella posizione indicata
- Implementare un metodo: float calcolaMedia() che calcoli la media dei voti dei corsi
- Implementare un metodo: void stampa() che visualizzi i dati dello studente e l'elenco dei corsi

Nel main(): chiedere all'utente:

- nome e cognome dello studente, numero di corsi
- Creare un oggetto Studente
- Per ogni corso:
  - chiedere nome del corso, chiedere voto, inserire i dati tramite il metodo setCorso
- Calcolare e stampare la media dei voti

[Esercizio C061] [*prova completa del 7 giugno 2017, esercizio 5*]

Un'azienda ha otto dipartimenti. I dipendenti che lavorano in ciascun dipartimento possono appartenere ad una di cinque diverse classi stipendiali.

Allo scopo di gestire i dipendenti dell'azienda, si realizzi in linguaggio C++ la classe *Dipendenti* avente i seguenti attributi:

- **un array _*s* di cinque numeri reali** per rappresentare lo stipendio corrispondente a ciascuna delle cinque classi stipendiali e

- **una matrice di numeri interi _D di otto righe e cinque colonne** i cui elementi rappresentano, per ciascun dipartimento, il numero di dipendenti appartenenti a ciascuna classe stipendiale. Se, ad esempio, l'elemento nella terza riga e quarta colonna di _D valesse 10, significherebbe che nel terzo dipartimento lavorano 10 dipendenti appartenenti alla quarta classe stipendiale.

[Esercizio C061] [*prova completa del 7 giugno 2017, esercizio 5*]

Si implementino, inoltre, i seguenti metodi (1/2):

- Il costruttore di default che inizializzi a zero tutti gli elementi dell'array _*s* e della matrice _D.

- Il costruttore con parametri che riceva come parametri un array di cinque numeri reali e una matrice di numeri interi di otto righe e cinque colonne e li assegni rispettivamente all'array _*s* e alla matrice _D. Per semplicità si supponga che i valori passati al costruttore siano sempre validi.

- Il costruttore di copia.

- Il distruttore.

- I selettori per l'array _*s* e per la matrice _D:
  - Il selettore per _*s* restituirà come valore di ritorno un puntatore a costante all'array stesso.
  - Il selettore per _D riceverà in ingresso (ovvero come parametri) due numeri interi *h* e *k* e restituirà come valore di ritorno l'elemento della matrice _D di indici *h* e *k* (un numero intero). Nel caso in cui il valore di almeno uno dei due parametri *h* e *k* non fosse valido, il metodo restituirà -1.

[Esercizio C061] [*prova completa del 7 giugno 2017, esercizio 5*]

Si implementino, inoltre, i seguenti metodi (2/2):

- I modificatori per l'array _*s* e per la matrice _D. Per semplicità, si supponga che l'array passato come parametro al modificatore per _*s* e la matrice passata come parametro al modificatore per _D abbiano le dimensioni corrette e che i loro elementi assumano valori validi.

- Il metodo *totaleDipendenti* che calcoli e restituisca come valore di ritorno il numero totale dei dipendenti dell'azienda (un numero intero dato dalla somma degli elementi della matrice _D)

- Il metodo *estremiStipendio* che calcoli e restituisca come parametri di uscita (ovvero passati per riferimento) gli indici dei due dipartimenti (due numeri interi) per i quali l'azienda spende l'ammontare massimo e l'ammontare minimo per gli stipendi dei dipendenti. Il metodo non restituisce alcun valore di ritorno.

[Esercizio C061] [*prova completa del 7 giugno 2017, esercizio 5*]

Si scriva quindi la funzione *main* che operi nel modo seguente:

- Dichiari un array *stp* di cinque numeri reali e una matrice di numeri interi Dip di otto righe e cinque colonne.
- Apra in lettura il file *Dati.txt*. Tale file contiene nella prima riga cinque numeri reali che rappresentano gli stipendi associati a ciascuna classe stipendiale e nelle otto righe successive, per ciascuna riga, cinque numeri interi che rappresentano il numero di dipendenti inquadrati in ciascuna classe stipendiale per il dipartimento corrispondente alla riga in esame. Per semplicità si supponga che i valori contenuti nel file siano validi. Nel caso in cui il file *Dati.txt* non esista, *main* termina con valore di ritorno -1.
- Legga la prima riga del file *Dati.txt* assegnandone i valori agli elementi dell'array *stp* e le righe successive assegnandone i valori agli elementi della matrice Dip.
- Utilizzando il costruttore con parametri, allochi dinamicamente un oggetto *pdip* di tipo *Dipendenti*, passando l'array *stp* e la matrice Dip come parametri al costruttore.
- Invochi su tale oggetto i metodi *totaleDipendenti* ed *estremiStipendio* e stampi a video il risultato.
- Deallochi l'oggetto *pdip* e termini.

[Esercizio C061] [*prova completa del 7 giugno 2017, esercizio 5*]

Esempio di file Dati.txt

20000 24000 28000 32000 36000
6 3 2 1 0
2 3 4 1 1
3 5 8 2 1
3 4 7 3 2
8 2 0 1 1
5 4 3 2 1
6 7 2 1 3
5 4 2 2 4